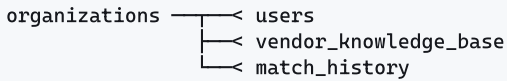


Database Schema

Four tables. `organizations` is the tenant root; `users`, `vendor_knowledge_base` and `match_history` all hang off it.



All models use the SQLAlchemy 2.0 style — `DeclarativeBase` with `Mapped[...]` / `mapped_column(...)`, not the legacy `Column()` declarative style.

Declarative base — `app/db/base.py`

The `type_annotation_map` is where the important decisions live. Setting them once here means no model ever has to remember that money is `NUMERIC(18,4)` or that timestamps carry a timezone.

```
from __future__ import annotations

import datetime as dt
import uuid
from decimal import Decimal
from typing import Any

from sqlalchemy import DateTime, MetaData, Numeric, String
from sqlalchemy.dialects.postgresql import JSONB, UUID
from sqlalchemy.orm import DeclarativeBase

# Deterministic constraint names, so Alembic autogenerate can actually DROP them
# instead of emitting a migration that references a name Postgres invented.
NAMING_CONVENTION = {
    "ix": "ix_(column_0_N_label)s",
    "uq": "uq_(table_name)s_(column_0_N_name)s",
    "ck": "ck_(table_name)s_(constraint_name)s",
    "fk": "fk_(table_name)s_(column_0_N_name)s_(referred_table_name)s",
    "pk": "pk_(table_name)s",
}

class Base(DeclarativeBase):
    metadata = MetaData(naming_convention=NAMING_CONVENTION)

    # One place to decide how Python types land in Postgres.
    type_annotation_map = {
        uuid.UUID: UUID(as_uuid=True),
        dt.datetime: DateTime(timezone=True),      # always TIMESTAMPTZ, never naive
        Decimal: Numeric(18, 4),                  # money: never float
        dict[str, Any]: JSONB,
        list[dict[str, Any]]: JSONB,
        str: String,
    }

    def __repr__(self) -> str: # pragma: no cover
        return f"<{type(self).__name__} id={getattr(self, 'id', None)}>"
```

Why `NUMERIC`, never `float`. An invoice total of `1234.56` stored as a float compares unequal to the same value read back, and the matching engine's tolerance bands then produce non-deterministic scores. `asyncpg` round-trips `Decimal` natively, so there is no conversion cost to paying attention here.

Mixins — app/models/mixins.py

```
from __future__ import annotations

import datetime as dt
import uuid

from sqlalchemy import DateTime, ForeignKey, func, text
from sqlalchemy.orm import Mapped, declared_attr, mapped_column

class UUIDPKMixin:
    id: Mapped[uuid.UUID] = mapped_column(
        primary_key=True,
        default=uuid.uuid4,
        server_default=text("gen_random_uuid()"), # built in from PostgreSQL 13
    )

class TimestampMixin:
    created_at: Mapped[dt.datetime] = mapped_column(
        DateTime(timezone=True), server_default=func.now(), nullable=False, index=True
    )
    updated_at: Mapped[dt.datetime] = mapped_column(
        DateTime(timezone=True),
        server_default=func.now(),
        onupdate=func.now(), # ORM-side only - add a DB trigger if you write raw SQL
        nullable=False,
    )

class OrgScopedMixin:
    """Every tenant-owned row. Declared as a mixin so the FK and its index can
    never be forgotten when someone adds a new table."""

    @declared_attr
    def organization_id(cls) -> Mapped[uuid.UUID]:
        return mapped_column(
            ForeignKey("organizations.id", ondelete="CASCADE"),
            nullable=False,
            index=True,
        )
```

Why UUID primary keys rather than serial integers. Invoice ids appear in URLs the user sees and shares (`/verify/{id}`). Sequential integers leak volume — a competitor can read your monthly invoice count off the id — and make cross-tenant enumeration attacks trivial to attempt.

organizations

The tenant root. It also owns the Odoo connection, because each customer has their own Odoo instance.

```
from __future__ import annotations

import datetime as dt
import enum
from typing import TYPE_CHECKING, Any

from sqlalchemy import Boolean, DateTime, Enum, String, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.db.base import Base
from app.models.mixins import TimestampMixin, UUIDPKMixin

if TYPE_CHECKING:
    from app.models.user import User
```

```

class OdooConnectionStatus(str, enum.Enum):
    NOT_CONFIGURED = "not_configured"
    OK = "ok"
    AUTH_FAILED = "auth_failed"
    UNREACHABLE = "unreachable"

class Organization(UUIDPKMixin, TimestampMixin, Base):
    """One tenant == one customer company == one Odoo instance."""

    __tablename__ = "organizations"

    name: Mapped[str] = mapped_column(String(255), nullable=False)
    slug: Mapped[str] = mapped_column(String(80), nullable=False, unique=True, index=True)
    is_active: Mapped[bool] = mapped_column(Boolean, nullable=False, default=True)

    # ---- Odoo connection (per tenant, NOT global env) -----
    odoo_url: Mapped[str | None] = mapped_column(String(500))
    odoo_db: Mapped[str | None] = mapped_column(String(255))
    odoo_username: Mapped[str | None] = mapped_column(String(255))
    # Fernet ciphertext. Never a plaintext column, never returned by any schema.
    odoo_api_key_encrypted: Mapped[str | None] = mapped_column(Text)
    odoo_uid_cache: Mapped[int | None] = mapped_column()
    odoo_status: Mapped[OdooConnectionStatus] = mapped_column(
        Enum(
            OdooConnectionStatus,
            name="odoo_connection_status",
            values_callable=lambda e: [m.value for m in e],
        ),
        nullable=False,
        default=OdooConnectionStatus.NOT_CONFIGURED,
    )
    odoo_last_verified_at: Mapped[dt.datetime | None] = mapped_column(
        DateTime(timezone=True)
    )

    # Per-tenant matching overrides: weights, tolerance bands, auto-confirm cutoff.
    settings: Mapped[dict[str, Any]] = mapped_column(default=dict, server_default="{}")

    users: Mapped[list["User"]] = relationship(
        back_populates="organization", cascade="all, delete-orphan", lazy="selectin"
    )

    @property
    def odoo_configured(self) -> bool:
        return all(
            [self.odoo_url, self.odoo_db, self.odoo_username, self.odoo_api_key_encrypted]
        )

```

`values_callable` on every Enum. Without it, SQLAlchemy persists the enum *member name* (`NOT_CONFIGURED`) rather than its *value* (`not_configured`). Since the API and the TypeScript union both speak the lowercase value, omitting this produces a mismatch that only surfaces at runtime.

Credential encryption — `app/core/crypto.py`

```
from __future__ import annotations

from functools import lru_cache

from cryptography.fernet import Fernet, InvalidToken

from app.core.config import settings
from app.core.errors import AppError

@lru_cache(maxsize=1)
def _fernet() -> Fernet:
    return Fernet(settings.CREDENTIAL_ENCRYPTION_KEY.get_secret_value().encode())

def encrypt_secret(plaintext: str) -> str:
    return _fernet().encrypt(plaintext.encode()).decode()

def decrypt_secret(ciphertext: str) -> str:
    try:
        return _fernet().decrypt(ciphertext.encode()).decode()
    except InvalidToken as exc:
        # Almost always means CREDENTIAL_ENCRYPTION_KEY was rotated or regenerated
        # while ciphertext encrypted under the old key is still in the database.
        raise AppError(
            "Stored Odoo credentials cannot be decrypted. Re-enter them in settings."
        ) from exc
```

users

```
from __future__ import annotations

import datetime as dt
import enum
from typing import TYPE_CHECKING

from sqlalchemy import Boolean, DateTime, Enum, Index, String
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.db.base import Base
from app.models.mixins import OrgScopedMixin, TimestampMixin, UUIDPKMixin

if TYPE_CHECKING:
    from app.models.organization import Organization

class UserRole(str, enum.Enum):
    OWNER = "owner"      # billing + Odoo credential management
    ADMIN = "admin"      # user + knowledge base management
    MEMBER = "member"    # upload + confirm invoices

class User(UUIDPKMixin, OrgScopedMixin, TimestampMixin, Base):
    __tablename__ = "users"
    __table_args__ = (
        Index("ix_users_org_active", "organization_id", "is_active"),
    )

    # Stored already lowercased by AuthService, so a plain unique constraint is
    # enough and we avoid a functional index Alembic cannot autogenerate.
    email: Mapped[str] = mapped_column(String(320), nullable=False, unique=True, index=True)
    hashed_password: Mapped[str] = mapped_column(String(255), nullable=False)
    full_name: Mapped[str | None] = mapped_column(String(255))
    role: Mapped[UserRole] = mapped_column(
        Enum(UserRole, name="user_role", values_callable=lambda e: [m.value for m in e]),
        nullable=False,
        default=UserRole.MEMBER,
    )
    is_active: Mapped[bool] = mapped_column(Boolean, nullable=False, default=True)
    last_login_at: Mapped[dt.datetime | None] = mapped_column(DateTime(timezone=True))

    organization: Mapped["Organization"] = relationship(
        back_populates="users", lazy="joined"
    )
```

`lazy="joined"` on `organization` is deliberate: `get_current_user` runs on every request and always needs the org for scoping. Loading it in the same round trip avoids an N+1 that would otherwise hit every single endpoint.

Password hashing — `app/core/security.py`

```
from __future__ import annotations

import datetime as dt
import uuid
from typing import Any, Literal

import jwt
from argon2 import PasswordHasher
from argon2.exceptions import InvalidHashError, VerifyMismatchError

from app.core.config import settings
from app.core.errors import AuthenticationError

# Argon2id with library defaults - the OWASP-recommended algorithm. Chosen over
# passlib+bcrypt: passlib 1.7.4 is unmaintained and has repeatedly broken against
# new Python and bcrypt releases.
_hasher = PasswordHasher()
```

```

# Pre-computed hash of a throwaway password. Verifying against this on an unknown
# email keeps login timing constant, so an attacker cannot enumerate valid
# accounts by measuring how fast the failure comes back.
_DUMMY_HASH = _hasher.hash("timing-attack-mitigation-placeholder")

def hash_password(plain: str) -> str:
    return _hasher.hash(plain)

def verify_password(plain: str, hashed: str | None) -> bool:
    try:
        _hasher.verify(hashed or _DUMMY_HASH, plain)
        return hashed is not None
    except (VerifyMismatchError, InvalidHashError):
        return False

def needs_rehash(hashed: str) -> bool:
    """True when argon2's default parameters have been raised since this hash was
    made. Call after a successful login and transparently upgrade the stored hash."""
    return _hasher.check_needs_rehash(hashed)

def create_token(
    subject: uuid.UUID,
    *,
    token_type: Literal["access", "refresh"] = "access",
    extra: dict[str, Any] | None = None,
) -> str:
    now = dt.datetime.now(dt.UTC)
    lifetime = (
        dt.timedelta(minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES)
        if token_type == "access"
        else dt.timedelta(days=settings.REFRESH_TOKEN_EXPIRE_DAYS)
    )
    payload = {
        "sub": str(subject),
        "typ": token_type,
        "iat": now,
        "exp": now + lifetime,
        "jti": uuid.uuid4().hex,
        **(extra or {}),
    }
    return jwt.encode(
        payload,
        settings.SECRET_KEY.get_secret_value(),
        algorithm=settings.JWT_ALGORITHM,
    )

def decode_token(token: str, *, expected_type: str = "access") -> dict[str, Any]:
    try:
        payload = jwt.decode(
            token,
            settings.SECRET_KEY.get_secret_value(),
            algorithms=[settings.JWT_ALGORITHM],
        )
    except jwt.ExpiredSignatureError as exc:
        raise AuthenticationError("Token expired.") from exc
    except jwt.InvalidTokenError as exc:
        raise AuthenticationError("Invalid token.") from exc

    # Without this check a refresh token would be accepted as an access token,
    # silently granting a 14-day session to any endpoint.
    if payload.get("typ") != expected_type:
        raise AuthenticationError(f"Expected a {expected_type} token.")
    return payload

```

vendor_knowledge_base

The learning half of the system. Every confirmed match teaches it one alias.

```

from __future__ import annotations

import datetime as dt
import enum
import uuid
from typing import Any

from sqlalchemy import (
    CheckConstraint, DateTime, Enum, ForeignKey, Index, Integer, Numeric,
    String, UniqueConstraint,
)
from sqlalchemy.orm import Mapped, mapped_column

from app.db.base import Base
from app.models.mixins import OrgScopedMixin, TimestampMixin, UUIDPKMixin

class AliasSource(str, enum.Enum):
    USER_CONFIRMED = "user_confirmed" # a human clicked confirm - highest trust
    AUTO_LEARNED = "auto_learned" # score >= AUTO_CONFIRM_THRESHOLD
    IMPORTED = "imported" # bulk seeded from Odoo res.partner names

class VendorKnowledgeBase(UUIDPKMixin, OrgScopedMixin, TimestampMixin, Base):
    """Learned mapping: whatever the OCR read as the vendor -> an Odoo partner id.

    The unique key is (organization_id, normalized_key), NOT the raw string.
    'ACME Corp.', 'acme corp' and 'ACME CORP' must collapse into one row whose
    hit_count means something. raw_vendor_string keeps the first-seen spelling for
    display and debugging; every other spelling lands in raw_variants.
    """

    __tablename__ = "vendor_knowledge_base"
    __table_args__ = (
        UniqueConstraint(
            "organization_id", "normalized_key", name="uq_vendor_kb_org_normalized_key"
        ),
        Index("ix_vendor_kb_org_partner", "organization_id", "odoo_partner_id"),
        Index("ix_vendor_kb_org_lastused", "organization_id", "last_used_at"),
        CheckConstraint("confidence >= 0 AND confidence <= 100", name="confidence_range"),
        CheckConstraint("hit_count >= 0", name="hit_count_non_negative"),
    )

    raw_vendor_string: Mapped[str] = mapped_column(String(500), nullable=False)
    normalized_key: Mapped[str] = mapped_column(String(500), nullable=False, index=True)
    # Every distinct raw spelling ever mapped here:
    # [{"raw": "ACME Corp.", "at": "2026-08-12T...", ...}]
    raw_variants: Mapped[list[dict[str, Any]]] = mapped_column(
        default=list, server_default="[]"
    )

    odoo_partner_id: Mapped[int] = mapped_column(Integer, nullable=False)
    odoo_partner_name: Mapped[str] = mapped_column(String(500), nullable=False)
    odoo_partner_vat: Mapped[str | None] = mapped_column(String(64))

    hit_count: Mapped[int] = mapped_column(
        Integer, nullable=False, default=0, server_default="0"
    )
    confidence: Mapped[float] = mapped_column(
        Numeric(5, 2), nullable=False, default=100.0, server_default="100.00"
    )
    source: Mapped[AliasSource] = mapped_column(
        Enum(AliasSource, name="alias_source", values_callable=lambda e: [m.value for m in e]),
        nullable=False,
        default=AliasSource.USER_CONFIRMED,
    )
    last_used_at: Mapped[dt.datetime | None] = mapped_column(DateTime(timezone=True))
    created_by_user_id: Mapped[uuid.UUID | None] = mapped_column(
        ForeignKey("users.id", ondelete="SET NULL")
    )

```

The unique constraint is the design. Keying on `normalized_key` rather than `raw_vendor_string` is what makes `hit_count` meaningful and makes `learn()` a single idempotent upsert instead of a read-modify-write race. The tradeoff: if you ever change `normalize_company_name()`, every stored key is invalidated, so ship a data migration that recomputes them.

match_history

One row per uploaded invoice. It is simultaneously the audit log, the work queue, and the training data for the knowledge base.

```
from __future__ import annotations

import datetime as dt
import enum
import uuid
from decimal import Decimal
from typing import Any

from sqlalchemy import (
    BigInteger, Date, DateTime, Enum, ForeignKey, Index, Integer, Numeric,
    String, Text,
)
from sqlalchemy.orm import Mapped, mapped_column

from app.db.base import Base
from app.models.mixins import OrgScopedMixin, TimestampMixin, UUIDPKMixin

class InvoiceStatus(str, enum.Enum):
    UPLOADED = "uploaded"          # blob stored, nothing processed yet
    PROCESSING = "processing"      # OCR / matching in flight
    OCR_FAILED = "ocr_failed"
    PENDING = "pending"           # matched, awaiting human verification
    CONFIRMED = "confirmed"        # human accepted a PO, not yet in Odoo
    REJECTED = "rejected"         # human said "no PO" / "not an invoice"
    PUSHING = "pushing"
    PUSHED = "pushed"             # vendor bill exists in Odoo
    PUSH_FAILED = "push_failed"   # confirmed, but Odoo write failed - retryable

class MatchMethod(str, enum.Enum):
    KB_ALIAS = "kb_alias"         # vendor resolved from the knowledge base
    FUZZY = "fuzzy"               # rapidfuzz on partner names
    MANUAL = "manual"             # user picked the PO by hand
    NONE = "none"

class MatchHistory(UUIDPKMixin, OrgScopedMixin, TimestampMixin, Base):
    __tablename__ = "match_history"
    __table_args__ = (
        # The work-queue query: org + status, newest first.
        Index("ix_match_history_org_status_created", "organization_id", "status", "created_at"),
        Index("ix_match_history_org_po", "organization_id", "matched_po_id"),
        # Duplicate-upload detection.
        Index("ix_match_history_org_hash", "organization_id", "file_sha256"),
        Index("ix_match_history_org_partner", "organization_id", "matched_partner_id"),
        Index("ix_match_history_org_invnum", "organization_id", "invoice_number"),
    )

    # ---- source document -----
    uploaded_by_user_id: Mapped[uuid.UUID | None] = mapped_column(
        ForeignKey("users.id", ondelete="SET NULL")
    )
    original_filename: Mapped[str] = mapped_column(String(500), nullable=False)
    storage_path: Mapped[str] = mapped_column(String(1000), nullable=False)
    mime_type: Mapped[str] = mapped_column(String(120), nullable=False)
    file_size_bytes: Mapped[int] = mapped_column(BigInteger, nullable=False)
    file_sha256: Mapped[str] = mapped_column(String(64), nullable=False)
    page_count: Mapped[int | None] = mapped_column(Integer)
```



```

status: Mapped[InvoiceStatus] = mapped_column(
    Enum(InvoiceStatus, name="invoice_status", values_callable=lambda e: [m.value for m in e]),
    nullable=False,
    default=InvoiceStatus.UPLOADED,
    index=True,
)

# ---- OCR -----
ocr_model: Mapped[str | None] = mapped_column(String(120))
ocr_duration_ms: Mapped[int | None] = mapped_column(Integer)
ocr_raw: Mapped[dict[str, Any] | None] = mapped_column() # JSONB: full response
ocr_markdown: Mapped[str | None] = mapped_column(Text) # concatenated pages
extracted: Mapped[dict[str, Any] | None] = mapped_column() # JSONB: ExtractedInvoice
line_items: Mapped[list[dict[str, Any]] | None] = mapped_column() # JSONB: denormalized

# ---- promoted scalars (indexed / filterable / sortable) -----
vendor_name: Mapped[str | None] = mapped_column(String(500))
invoice_number: Mapped[str | None] = mapped_column(String(120))
invoice_date: Mapped[dt.date | None] = mapped_column(Date)
due_date: Mapped[dt.date | None] = mapped_column(Date)
currency: Mapped[str | None] = mapped_column(String(3))
subtotal_amount: Mapped[Decimal | None] = mapped_column(Numeric(18, 4))
tax_amount: Mapped[Decimal | None] = mapped_column(Numeric(18, 4))
total_amount: Mapped[Decimal | None] = mapped_column(Numeric(18, 4))

# ---- matching -----
matched_partner_id: Mapped[int | None] = mapped_column(Integer)
matched_po_id: Mapped[int | None] = mapped_column(Integer)
matched_po_name: Mapped[str | None] = mapped_column(String(120))
match_score: Mapped[Decimal | None] = mapped_column(Numeric(5, 2))
match_method: Mapped[MatchMethod] = mapped_column(
    Enum(MatchMethod, name="match_method", values_callable=lambda e: [m.value for m in e]),
    nullable=False,
    default=MatchMethod.NONE,
)
score_breakdown: Mapped[dict[str, Any] | None] = mapped_column() # JSONB per-component
candidates: Mapped[list[dict[str, Any]] | None] = mapped_column() # JSONB top-N snapshot
matching_duration_ms: Mapped[int | None] = mapped_column(Integer)

# ---- human decision -----
confirmed_by_user_id: Mapped[uuid.UUID | None] = mapped_column(
    ForeignKey("users.id", ondelete="SET NULL")
)
confirmed_at: Mapped[dt.datetime | None] = mapped_column(DateTime(timezone=True))
# [{"field": "total_amount", "from": "1200.00", "to": "1250.00",
#   "at": "2026-08-12T...", "by": "<user uuid>"}]
user_corrections: Mapped[list[dict[str, Any]]] = mapped_column(
    default=list, server_default="[]"
)
rejection_reason: Mapped[str | None] = mapped_column(Text)
alias_learned: Mapped[bool] = mapped_column(default=False, server_default="false")

# ---- odoo write-back -----
odoo_bill_id: Mapped[int | None] = mapped_column(Integer)
odoo_bill_name: Mapped[str | None] = mapped_column(String(120))
pushed_at: Mapped[dt.datetime | None] = mapped_column(DateTime(timezone=True))
push_attempts: Mapped[int] = mapped_column(
    Integer, nullable=False, default=0, server_default="0"
)
error_code: Mapped[str | None] = mapped_column(String(80))
error_message: Mapped[str | None] = mapped_column(Text)

```

Why both JSONB and promoted scalar columns

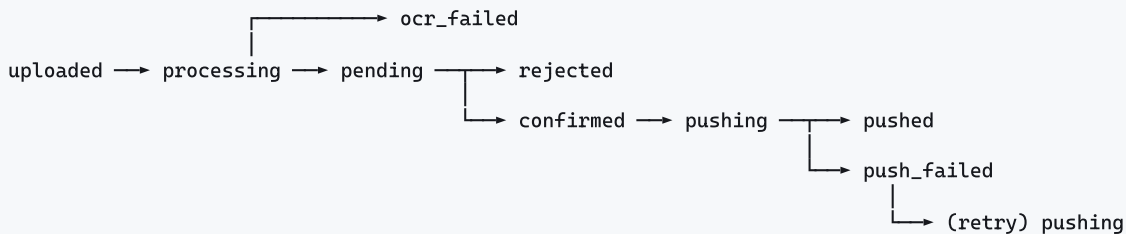
`extracted` holds the complete OCR payload, and `vendor_name` / `total_amount` / `invoice_date` duplicate three of its fields as real columns. That redundancy is deliberate:

- **JSONB** keeps everything, including fields you have not thought to use yet, and lets you re-run the matching engine against historical data after tuning it.
- **Promoted columns** are what you can index, filter, sort and aggregate. `WHERE vendor_name ILIKE ...` on a real column with a real index is orders of magnitude faster than the same query against a JSONB path,

and the dashboard's status/date filters depend on it.

The promoted columns are also the ones the user edits during correction, so they are the system of record for the confirmed values, while `extracted` preserves what the OCR originally said. Keeping both is what makes the `user_corrections` audit trail meaningful.

The status lifecycle



`push_failed` is a distinct state rather than a rollback to `pending`, and that distinction carries real weight. When the human has confirmed a match but Odoo is unreachable, the decision is already valid — the alias has been learned and the correction recorded. Only the external write needs retrying. Collapsing this into `pending` would ask the user to redo work they already did.

Migration notes

Generate the first revision:

```
cd C:\ocr\server
..\venv\Scripts\alembic.exe revision --autogenerate -m "init schema"
..\venv\Scripts\alembic.exe upgrade head
```

Before running it, edit the generated revision:

1. Add `op.execute("CREATE EXTENSION IF NOT EXISTS pgcrypto")` as the first statement in `upgrade()`.
2. Confirm the five enum types are created: `odoo_connection_status`, `user_role`, `alias_source`, `invoice_status`, `match_method`.
3. Check the `CheckConstraint`s on `vendor_knowledge_base` survived autogeneration — Alembic is inconsistent about picking these up.

Verify:

```
dt -- 4 tables + alembic_version
dT -- 5 enum types
d match_history -- 6 indexes, all prefixed organization_id
SELECT enum_range(NULL::invoice_status); -- 9 values
```

Adding an enum value later requires a hand-written migration; Alembic never autogenerates `ALTER TYPE`:

```
def upgrade() -> None:
    op.execute("ALTER TYPE invoice_status ADD VALUE IF NOT EXISTS 'archived'")
```